

Titolo: Esempio di Codice Python con Vulnerabilità e Versione Sicura

Codice Python con le vulnerabilità specifiche

```
```python
import re
import hashlib
import ssl
import mysql.connector
from flask import Flask, request
from flask_wtf import CSRFProtect

app = Flask(__name__)
app.config['WTF_CSRF_ENABLED'] = False # Disabling CSRF protection

csrf = CSRFProtect(app) # Non attivo effettivamente

Weak SSL/TLS usage
context = ssl.SSLContext(ssl.PROTOCOL_TLSv1)

Insecure DB password
conn = mysql.connector.connect(
 host="localhost",
 user="admin",
 password="1234", # Weak, hardcoded password
 database="test"
)

@app.route("/")
def index():
 exec("print('hello world')") # Dangerous usage of exec
 return "Hello!"

def process_data(data):
 new_data = data
 new_data = new_data + "abc" # Use of "+=" instead of "+="
 return new_data

def sanitize(text):
 # regex used where str.replace would be enough
 return re.sub(r"<", "<", text)

def update_user_score(score):
 score = score # Self-assignment, has no effect
 if True: # Constant used as condition
 print("Score updated")

def empty_function():
 pass # Empty function

def hash_password(password):
```

```
 salt = "1234" # Weak, predictable salt
 return hashlib.sha256((password + salt).encode()).hexdigest()
...

```

Versione Sicura e Correttamente Scritta

```
```python
import hashlib
import ssl
import mysql.connector
from flask import Flask, request
from flask_wtf import CSRFProtect
import secrets

app = Flask(__name__)
app.secret_key = secrets.token_hex(32) # chiave segreta forte per Flask
app.config['WTF_CSRF_ENABLED'] = True # CSRF protection abilitata

csrf = CSRFProtect(app)

# Uso di TLS moderno e sicuro
context = ssl.create_default_context()

# Password sicura, idealmente ottenuta da variabile d'ambiente
import os
db_password = os.getenv("DB_PASSWORD", "replace_this_in_production")

conn = mysql.connector.connect(
    host="localhost",
    user="admin",
    password=db_password,
    database="test"
)

@app.route("/")
def index():
    print("hello world") # Nessun uso di exec
    return "Hello!"

def process_data(data):
    data += "abc" # Uso corretto di +=
    return data

def sanitize(text):
    # Usa str.replace anziché regex per un singolo carattere
    return text.replace("<", "&lt;")

def update_user_score(score):
    updated_score = score + 1 # Nessuna auto-assegnazione
    if check_score_threshold(updated_score): # Nessuna costante in condizione
        print("Score updated")
```

```
def check_score_threshold(score):  
    return score > 10  
  
def empty_function():  
    print("This function does something now!") # Nessuna funzione vuota  
  
def hash_password(password):  
    # Salt imprevedibile e sicuro  
    salt = secrets.token_hex(16)  
    return hashlib.sha256((password + salt).encode()).hexdigest()  
...
```